.UBAfiuba
FACULTAD DE INGENIERÍA

Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

Trabajo Práctico 1
Los Algoritmos Greedy son juegos de niños

12 de octubre de 2026

Guadalupe Schell
114335

Abril Erin Borsnich
114760

Índice

1. Análisis del algoritmo **3**
1.1. Enunciado del problema . . . 3
1.2. Análisis del problema y algoritmo planteado . . . 3
1.3. Ejemplos de ejecución . . . 3
1.3.1. Ejemplo 1 . . . 3
1.3.2. Ejemplo 2 . . . 4
1.3.3. Ejemplo 3: instancia correspondiente a la excepción . . . 4

2. Optimalidad del algoritmo **4**
2.1. Demostración . . . 4
2.2. Relación entre las elecciones de Sophia y Mateo . . . 5
2.3. Caso de cantidad impar de monedas . . . 5
2.4. Caso de cantidad par de monedas . . . 5
2.5. Consideración de la excepción del enunciado . . . 6

3. Implementación del algoritmo **6**
3.1. Código funcional . . . 6
3.2. Complejidad . . . 7
3.3. Análisis de variabilidad en los valores de las monedas . . . 7
3.3.1. Impacto en los tiempos del algoritmo . . . 7
3.3.2. Impacto en la optimalidad . . . 7
3.4. Mediciones de tiempo . . . 7
3.5. Conclusión . . . 9

A. Análisis experimental de la magnitud y la variabilidad de los valores **10**
A.1. Generación de las instancias y metodología experimental . . . 10
A.2. Influencia de la magnitud de los valores . . . 10
A.3. Influencia de la variabilidad de los valores . . . 11
A.4. Conclusiones del análisis experimental . . . 11

1. Análisis del algoritmo

1.1. Enunciado del problema

Sophia y Mateo juegan a un juego utilizando una fila de n monedas, donde cada moneda tiene asociado un valor. En cada turno solamente puede retirarse una de las dos monedas que se encuentran en los extremos de la fila. Al elegirla, la remueve de la fila, y le toca luego al otro jugador, quien debe elegir otra moneda siguiendo la misma regla. Siguen agarrando monedas hasta que no quede ninguna. Quien gane será quien tenga el mayor valor acumulado (por sumatoria).

Debido a que Mateo todavía es demasiado pequeño para comprender el juego, es Sophia quien debe elegir las monedas que Mateo obtiene. Sin embargo, Sophia es competitiva y busca asegurarse de ganar el juego.

Sabiendo que Sophia comienza siempre el juego, el objetivo es determinar, dada una secuencia de valores de monedas, qué moneda debe elegir Sophia en cada turno para ella y qué moneda debe elegir para Mateo, de manera que Sophia siempre garantice su victoria.

Se solicita hacer un análisis del problema y proponer un algoritmo greedy que obtenga la solución óptima al problema planteado, desestimando el caso de una cantidad par de monedas en el que cada valor aparece la misma cantidad de veces, ya que en ese caso puede producirse un empate independientemente de la estrategia de Sophia.

1.2. Análisis del problema y algoritmo planteado

Tenemos una fila de n monedas y, en cada turno, el jugador debe elegir una de las dos monedas que se encuentran en los extremos. El objetivo es encontrar la manera en que Sophia siempre gane el juego, respetando las reglas y el turno de su hermano.

Se propone el siguiente algoritmo greedy:

- Cuando corresponde elegir una moneda para Sophia, se toma la moneda de mayor valor entre los dos extremos disponibles.
- Cuando corresponde elegir una moneda para Mateo, Sophia elige para él la moneda de menor valor entre los dos extremos disponibles.

En otras palabras, si los extremos actuales son izq y der, Sophia elige $\max(izq, der)$ y, posteriormente, Mateo recibe $\min(izq, der)$.

Regla greedy. En cada iteración se elige primero el extremo de mayor valor para Sophia y luego el de menor valor para Mateo. En cada turno se considera únicamente el estado actual de la fila, sin necesidad de conocer las elecciones pasadas o prever elecciones futuras.

Óptimo local. Para Sophia, el óptimo local consiste en quedarse con el mayor valor disponible en ese momento. Para Mateo, desde la perspectiva de Sophia, el óptimo local consiste en asignarle el menor valor disponible.

Por lo tanto, en cada ronda, Sophia obtiene la mejor de las dos opciones disponibles para ella y luego le asigna a Mateo la peor de las opciones que quedan. Esto es lo que luego permitirá demostrar la optimalidad del algoritmo.

1.3. Ejemplos de ejecución

1.3.1. Ejemplo 1

Consideremos la siguiente fila de monedas: $[7, 20, 30, 10, 9]$. Sophia comienza.

Paso 1. Los extremos son 7 y 9. Sophia toma el mayor: $S_1 = 9$. La fila queda $[7, 20, 30, 10]$. Ahora Sophia debe elegir la moneda para Mateo. Los extremos son 7 y 10. Para Mateo se elige el menor: $M_1 = 7$. La fila queda $[20, 30, 10]$.

Paso 2. Los extremos son 20 y 10. Sophia toma $S_2 = 20$. Queda $[30, 10]$. Para Mateo se elige $M_2 = 10$. Finalmente queda $[30]$. Sophia obtiene la última moneda: $S_3 = 30$.

Resultado. Sophia obtiene $9 + 20 + 30 = 59$. Mateo obtiene $7 + 10 = 17$. Por lo tanto, $59 > 17$ y Sophia gana.

1.3.2. Ejemplo 2

Consideremos ahora una instancia con valores repetidos: $[4, 4, 4, 7, 7, 7, 9]$. El valor 4 aparece 3 veces, el valor 7 aparece 3 veces y el valor 9 aparece una vez. Ningún valor aparece una cantidad par de veces.

Paso 1. Los extremos son 4 y 9. Sophia toma el mayor: $S_1 = 9$. La fila queda $[4, 4, 4, 7, 7, 7]$. Para Mateo se elige el menor entre los extremos, $M_1 = 4$. La fila queda $[4, 4, 7, 7, 7]$.

Paso 2. Los extremos son 4 y 7. Sophia toma $S_2 = 7$. Queda $[4, 4, 7, 7]$. Para Mateo se elige $M_2 = 4$. Queda $[4, 7, 7]$.

Paso 3. Sophia toma 7 y la fila queda $[4, 7]$. Para Mateo se elige 4 y queda $[7]$. Sophia obtiene la última moneda, de valor 7.

Entonces, $S = 9 + 7 + 7 + 7 = 30$ y $M = 4 + 4 + 4 = 12$. Por lo tanto, $30 > 12$ y Sophia gana. Este ejemplo permite observar que la estrategia no requiere que todos los valores sean diferentes.

1.3.3. Ejemplo 3: instancia correspondiente a la excepción

Consideremos $[1, 2, 2, 1]$. El valor 1 aparece dos veces y el valor 2 también aparece dos veces. Por lo tanto, esta instancia está dentro de la excepción contemplada en el enunciado.

Aplicando el algoritmo, Sophia toma una moneda de valor 1, Mateo recibe la otra moneda de valor 1, Sophia toma una moneda de valor 2 y Mateo recibe la restante. Entonces, $S = 1 + 2 = 3$ y $M = 1 + 2 = 3$. Se produce un empate: $S = M$.

Debido a que este caso se encuentra explícitamente fuera del conjunto de instancias para las cuales se exige demostrar que Sophia siempre gana, no contradice la demostración de optimalidad.

2. Optimalidad del algoritmo

2.1. Demostración

En el contexto de esta consigna, consideramos óptima una estrategia que garantiza la victoria de Sophia. La demostración a continuación prueba precisamente esa propiedad.

Queremos demostrar que la estrategia propuesta garantiza que Sophia gana para cualquier instancia válida del problema, es decir, para cualquier cantidad de monedas y cualquier orden de valores, siempre que no se presente la excepción indicada en el enunciado.

Sean $S_1, S_2, \ldots$ los valores de las monedas que obtiene Sophia y $M_1, M_2, \ldots$ los valores de las monedas que recibe Mateo. Consideremos una ronda cualquiera i. Sean a y b los valores de las dos monedas que se encuentran en los extremos al comenzar la ronda. Suponiendo $a \geq b$, por la regla greedy, Sophia elige la moneda a, por lo que $S_i = a$.

Luego quedan disponibles la moneda b junto con la nueva moneda que quedó en el extremo opuesto. Sophia debe elegir una moneda para Mateo y aplica nuevamente la estrategia, seleccionando la menor de las dos disponibles. Por lo tanto, $M_i \leq b$. Como además $b \leq a$, se obtiene:

$$M_i \leq b \leq a = S_i.$$

Por lo tanto, en cada ronda Sophia obtiene una moneda cuyo valor es mayor o igual al valor de la moneda que recibe Mateo. Es decir, en cada ronda se cumple $M_i \leq S_i$.

2.2. Relación entre las elecciones de Sophia y Mateo

Sumando la desigualdad obtenida para todas las rondas:

$$S_1 + S_2 + \cdots + S_k \geq M_1 + M_2 + \cdots + M_k.$$

Por lo tanto, $S \geq M$ si ambos jugadores tienen la misma cantidad de monedas. Es decir, Sophia nunca puede terminar con un valor total menor que Mateo. Pero la consigna no pide únicamente que Sophia no pierda: pide que gane. Para obtener la desigualdad estricta analizamos por separado la cantidad de monedas.

2.3. Caso de cantidad impar de monedas

Supongamos que $n = 2k + 1$. Como Sophia comienza y los turnos se alternan, Sophia obtiene $k + 1$ monedas, mientras que Mateo obtiene k monedas. Por cada una de las primeras k rondas se cumple $S_i \geq M_i$. Además, Sophia obtiene una moneda adicional al final. Como los valores de las monedas son positivos, esta moneda adicional tiene valor estrictamente mayor que cero.

$$S = \sum_{i=1}^{k} S_i + S_{k+1}, \qquad M = \sum_{i=1}^{k} M_i.$$

Como $S_i \geq M_i$ para cada ronda y $S_{k+1} > 0$, se obtiene $S > M$. Por lo tanto, si n es impar, Sophia siempre gana.

2.4. Caso de cantidad par de monedas

Ahora supongamos que $n = 2k$. En este caso Sophia y Mateo obtienen la misma cantidad de monedas. Por la propiedad demostrada anteriormente, $S_i \geq M_i$ para cada ronda. Por lo tanto, $S \geq M$.

Para demostrar que Sophia gana, debemos descartar la posibilidad de que $S = M$. Supongamos, por contradicción, que $S = M$. Como en todas las rondas se cumple $S_i \geq M_i$, y la suma total de ambos jugadores es igual, necesariamente debe cumplirse igualdad en todas las rondas: $S_i = M_i$ para todo i.

Es decir, cada ronda debe aportar exactamente el mismo valor a Sophia y a Mateo. Como cada moneda es asignada exactamente una vez, esto implica que las monedas pueden agruparse en pares de igual valor:

$$(S_1, M_1), (S_2, M_2), \ldots, (S_k, M_k),$$

con $S_i = M_i$. Por lo tanto, para cada valor que aparezca en la instancia, la cantidad total de monedas de dicho valor debe ser par. En consecuencia:

$$S = M \implies \text{todo valor aparece una cantidad par de veces.}$$

Pero este es precisamente el caso excluido por el enunciado. Por lo tanto, en las instancias válidas no puede ocurrir $S = M$. Como ya demostramos que $S \geq M$, la única posibilidad restante es $S > M$ y, por lo tanto, Sophia gana.

2.5. Consideración de la excepción del enunciado

La excepción resulta necesaria. Si existe una instancia con una cantidad par de monedas de cada valor, puede suceder que Sophia y Mateo terminen obteniendo exactamente el mismo valor total. Por ejemplo, $[1, 2, 2, 1]$ produce $S = 3$ y $M = 3$. Por lo tanto, no sería correcto afirmar que Sophia puede garantizar una victoria estricta para absolutamente cualquier instancia.

En consecuencia, la demostración completa puede resumirse de la siguiente manera:

- En cada ronda: $S_i \geq M_i$.
- Si n es impar, Sophia obtiene una moneda adicional positiva y, por lo tanto, $S > M$.
- Si n es par, para que ocurra un empate deberían cumplirse igualdades en todas las rondas. Eso implica que cada valor aparece una cantidad par de veces. Como esas instancias están excluidas por la consigna, en cualquier instancia válida con n par se cumple $S > M$.

Por lo tanto, el algoritmo greedy garantiza que Sophia gana en toda instancia válida.

3. Implementación del algoritmo

3.1. Código funcional

La implementación mantiene dos indices, uno apuntando al extremo izquierdo de la fila y otro al extremo derecho. En cada iteracion, Sophia toma el mayor de los dos extremos y se mueve el índice correspondiente. Si todavia quedan monedas, Sophia elige para Mateo el menor de los extremos restantes.

El algoritmo registra en una lista las decisiones tomadas para poder informar explícitamente qué extremo se asigna a cada jugador, tal como requiere la consigna.

```python
def resolver(monedas):
    izquierda = 0
    derecha = len(monedas) - 1
    decisiones = []
    ganancia_sophia = 0
    turno_sophia = True

    while izquierda <= derecha:
        if turno_sophia:
            # Sophia elige la moneda de mayor valor disponible.
            if monedas[izquierda] >= monedas[derecha]:
                ganancia_sophia += monedas[izquierda]
                decisiones.append("Primera moneda para Sophia")
                izquierda += 1
            else:
                ganancia_sophia += monedas[derecha]
                decisiones.append("Ultima moneda para Sophia")
                derecha -= 1
        else:
            # Sophia asigna a Mateo la moneda de menor valor disponible.
            if monedas[izquierda] <= monedas[derecha]:
                decisiones.append("Primera moneda para Mateo")
                izquierda += 1
            else:
                decisiones.append("Ultima moneda para Mateo")
                derecha -= 1
        turno_sophia = not turno_sophia

    return decisiones, ganancia_sophia
```

3.2. Complejidad

Sea n la cantidad de monedas de la entrada. El algoritmo utiliza dos indices: *izquierda* y *derecha*. En cada turno se retira una moneda y uno de los indices se mueve una posición. Como cada moneda se procesa exactamente una vez, el ciclo realiza n iteraciones en total.

En cada iteración se realizan únicamente operaciones de tiempo constante: comparación, inserción, incremento de un índice y suma. Por lo tanto, el costo de procesar una moneda es $O(1)$. Para n monedas:

$$T(n) = \sum_{i=1}^{n} O(1) = O(n).$$

A diferencia de otros algoritmos greedy vistos en la materia, no es necesario ordenar previamente la entrada, lo que evita el costo típico de $O(n \log n)$ asociado al ordenamiento.

3.3. Análisis de variabilidad en los valores de las monedas

3.3.1. Impacto en los tiempos del algoritmo

La variabilidad de los valores de las monedas no afecta el tiempo de ejecución del algoritmo.

Al utilizar dos punteros (`izquierda` y `derecha`), el algoritmo siempre realiza exactamente n iteraciones. En cada paso solo se hacen comparaciones e incrementos de índices, que son operaciones de costo constante $O(1)$.

A diferencia de lo que ocurre en problemas con multiplicaciones, donde el número de dígitos crece exponencialmente y el costo de operar deja de ser constante, al sumar n valores, la cantidad de bits del resultado final crece de forma insignificante, por lo que no impacta en la práctica. Por ende, el tiempo se mantiene lineal ($O(n)$), sin importar las magnitudes de las monedas.

3.3.2. Impacto en la optimalidad

La variabilidad de los valores no altera la optimalidad del algoritmo. La estrategia no depende de las magnitudes ni de la disposición de los datos, sino de tomar la decisión local óptima entre las dos puntas disponibles en cada turno.

Al elegir siempre la opción mayor para Sophia (S_i) y la menor para Mateo (M_i), se garantiza la condición $S_i \geq M_i$ en cada ronda. La única excepción donde el algoritmo no logra una victoria estricta ($S > M$) ocurre en instancias simétricas donde las frecuencias de los valores son pares y están distribuidas de tal forma que fuerzan un empate ($S = M$).

Salvo por dicho caso de empate, que se desestimó en la consigna, la optimalidad de la estrategia se mantiene invariante ante cualquier combinación de valores de las monedas.

3.4. Mediciones de tiempo

El algoritmo tiene una complejidad de $O(n)$, donde n es la cantidad de monedas. La función utiliza dos punteros, izquierda y derecha, que avanzan hacia el centro de forma lineal, realizando exactamente n iteraciones con operaciones de tiempo constante $O(1)$ en cada paso.

Se definieron 20 tamaños de muestra distribuidos equitativamente en el rango $n \in [100, 500{,}000]$ utilizando $np.linspace(100, 500{,}000, 20).astype(int)$. Para cada tamaño n, se generaron arreglos con valores de monedas aleatorios en el rango $[0, 100{,}000]$ mediante np.random.randint(0, 100.000, n).

Para verificar la hipótesis se midieron los tiempos de ejecución mediante timealgorithm y se realizó un ajuste por el método de cuadrados mínimos utilizando la función modelo lineal:

$$f(n) = a \cdot n + b$$

A partir de las mediciones se obtuvieron los parámetros óptimos a y b con la función scipy.optimize.curvefit.

Gráfico 1: Tiempo de ejecución vs. Ajuste Teórico

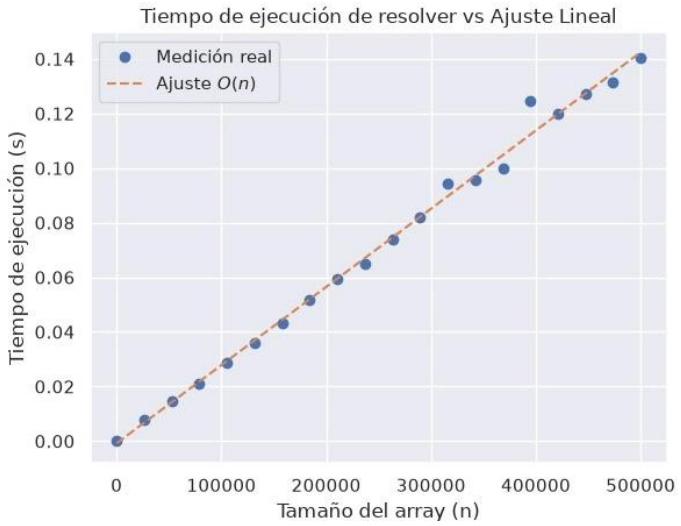
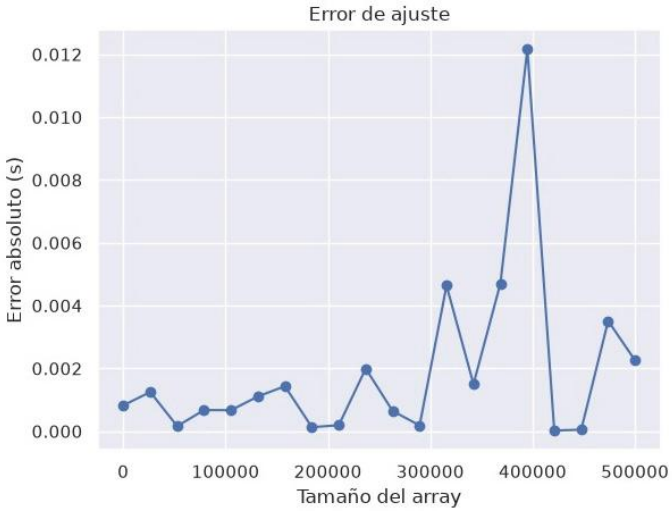


Figura 1: Error cuadrático total: 0.00022221509478733513

Gráfico 2: Error de Ajuste



Las mediciones temporales obtenidas confirman empíricamente la hipótesis teórica de complejidad $O(n)$. Como se observa en la figura del ajuste, los tiempos medidos se alinean sobre la función lineal $a \cdot n + b$. Asimismo, el error absoluto entre la predicción teórica y las mediciones se mantiene en un orden de magnitud despreciable, lo cual valida el comportamiento lineal del algoritmo

3.5. Conclusión

A lo largo de este trabajo se analizó el problema de determinar las elecciones óptimas de monedas para Sophia y Mateo a partir de una fila de n monedas, con el objetivo de garantizar la victoria de Sophia. Se propuso un algoritmo greedy que en cada turno selecciona la moneda de mayor valor disponible en los extremos para Sophia y le asigna la de menor valor a Mateo.

Este criterio se basa en maximizar en cada ronda la ganancia local de Sophia al tiempo que se minimiza la de su Mateo. De esta forma, se asegura que en cada iteración la suma acumulada por Sophia sea siempre mayor o igual a la de Mateo ($S_i \geq M_i$).

La optimalidad del algoritmo fue demostrada analizando formalmente los casos según la paridad de n. Se probó que para una cantidad impar de monedas la victoria es estricta gracias a la última moneda adicional, mientras que para un número par de monedas la única posibilidad de empate ocurre cuando todos los valores aparecen un número par de veces distribuídos simétricamente, caso excluido de la consigna. Esto permite concluir que el algoritmo greedy propuesto garantiza una solución óptima en toda instancia válida.

En cuanto a la complejidad, se determinó que el algoritmo tiene un costo lineal $O(n)$ gracias al uso de dos punteros que recorren la fila desde los extremos hacia el centro en un único pasaje. La validez de esta complejidad teórica fue confirmada empíricamente mediante mediciones de tiempo y un ajuste por el método de cuadrados mínimos.

Por otro lado, el análisis de variabilidad mostró que la magnitud de los valores de las monedas no afecta ni la complejidad temporal ni la optimalidad del algoritmo, manteniéndose el comportamiento lineal y la garantía de victoria de Sophia de manera invariante ante cualquier distribución de valores.

En conclusión, el algoritmo propuesto permite determinar las elecciones de monedas que aseguran la victoria de Sophia para cualquier instancia válida del problema.

A. Análisis experimental de la magnitud y la variabilidad de los valores

Anexo con las correciones sugeridas.

A.1. Generación de las instancias y metodología experimental

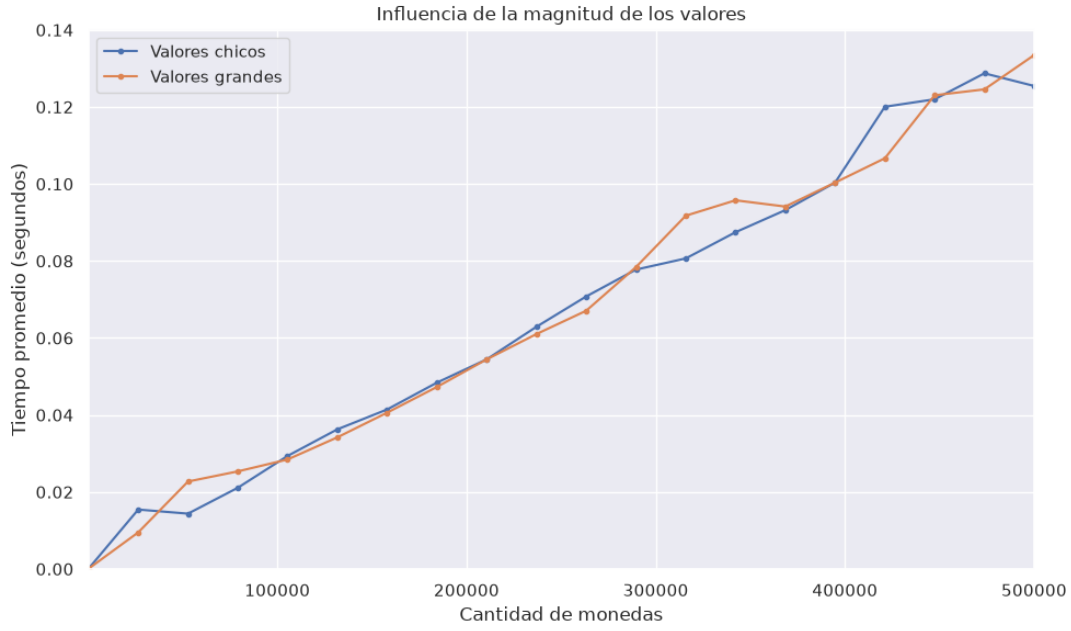
Para evaluar el comportamiento temporal del algoritmo, se generaron secuencias de monedas de distintos tamaños utilizando la función numpy.random.randint. Se consideraron 20 tamaños de entrada distribuidos uniformemente entre 100 y 500.000 monedas, mediante np.linspace(100, $500_000, 20).astype(int).Se fijólasemilladelgeneradordenúmerosaleatoriosen12345mediantenp.random.seed(12345)p$

- **Valores chicos:** enteros entre 1 y 100.
- **Valores grandes:** enteros entre 1.000.000 y 1.000.100.
- **Baja variabilidad:** enteros entre 499.995 y 500.005.
- **Alta variabilidad:** enteros entre 1 y 1.000.000.

Se entrega junto a este informe el codigo utilizado para realizar las mediciones.

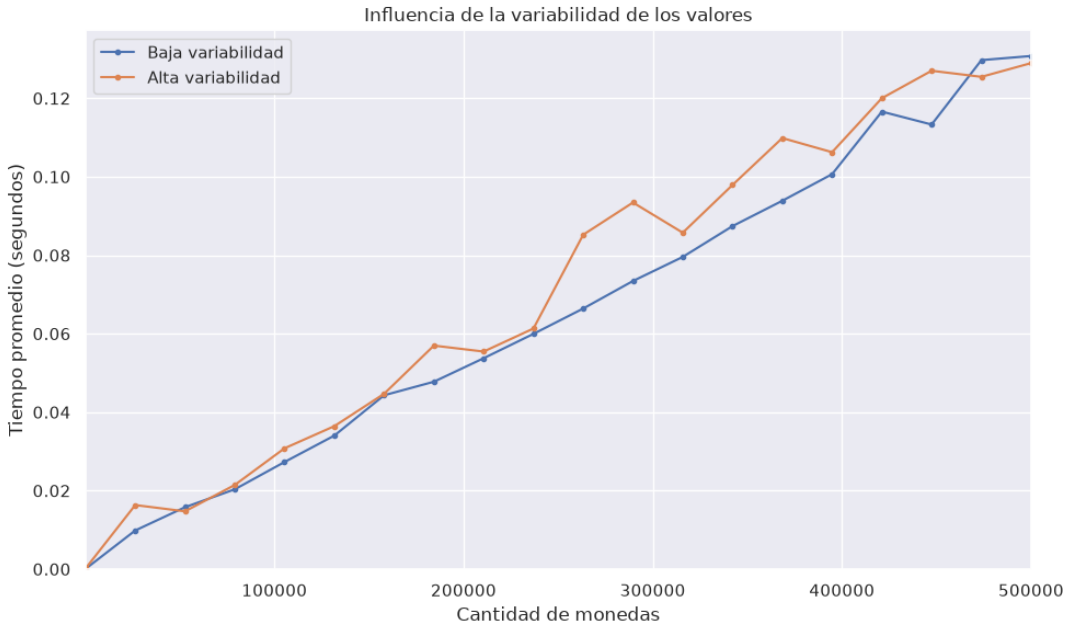
A.2. Influencia de la magnitud de los valores

Para analizar la influencia de la magnitud de los valores de las monedas, se compararon los tiempos de ejecución obtenidos con secuencias cuyos valores se encuentran entre 1 y 100 y secuencias cuyos valores se encuentran entre 1.000.000 y 1.000.100. En el gráfico se observa que ambas curvas presentan una tendencia general creciente a medida que aumenta la cantidad de monedas. Si bien existen diferencias entre algunas mediciones, las curvas se mantienen cerca una de otra. Por lo tanto, los resultados obtenidos muestran que la magnitud de los valores no afecta significativamente el tiempo de ejecución del algoritmo.



A.3. Influencia de la variabilidad de los valores

Para estudiar la influencia de la variabilidad de los valores, se compararon secuencias cuyos elementos se encuentran concentrados alrededor de 500.000 con secuencias que presentan una distribución más amplia, entre 1 y 1.000.000. En el gráfico se observa que ambas curvas mantienen una tendencia de crecimiento similar. Aunque se presentan diferencias no se produce una diferencia significativa en el tiempo de ejecución entre la dispersión de los valores y el tiempo de ejecución.



A.4. Conclusiones del análisis experimental

Los resultados son compatibles con el análisis teórico, según el cual el algoritmo realiza un recorrido de la secuencia y efectúa una cantidad constante de operaciones por moneda. Por este motivo, su complejidad temporal es (O(n)), independientemente de la magnitud y la variabilidad de los valores de las monedas.